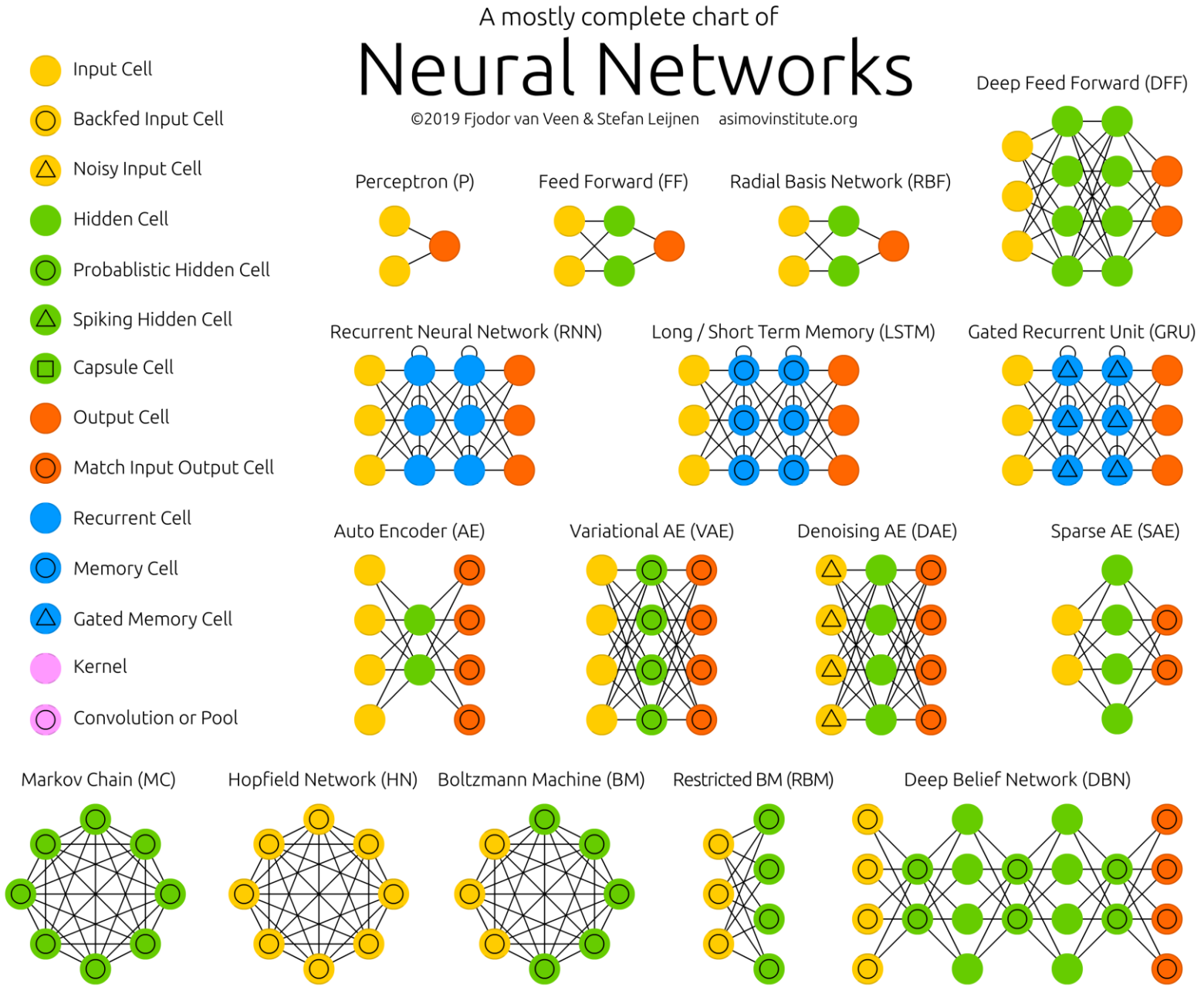


DL Frameworks y Aplicaciones

Rafael Pastor Vargas
SCC-ETSI Informática-UNED

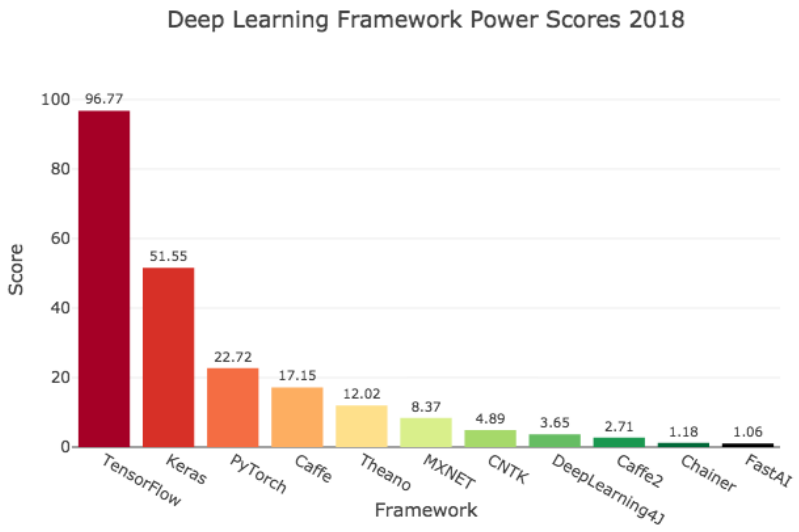
- http://www.asimovinstitute.org/neural-network-zoo/



DEEP LEARNING FRAMEWORKS

- https://blog.thedataincubator.com/2017/10/ranking-popular-deep-learning-libraries-for-data-science/
- https://towardsdatascience.com/deep-learning-framework-power-scores-2018-23607ddf297a
- https://www.predictiveanalyticstoday.com/deep-learning-software-libraries/

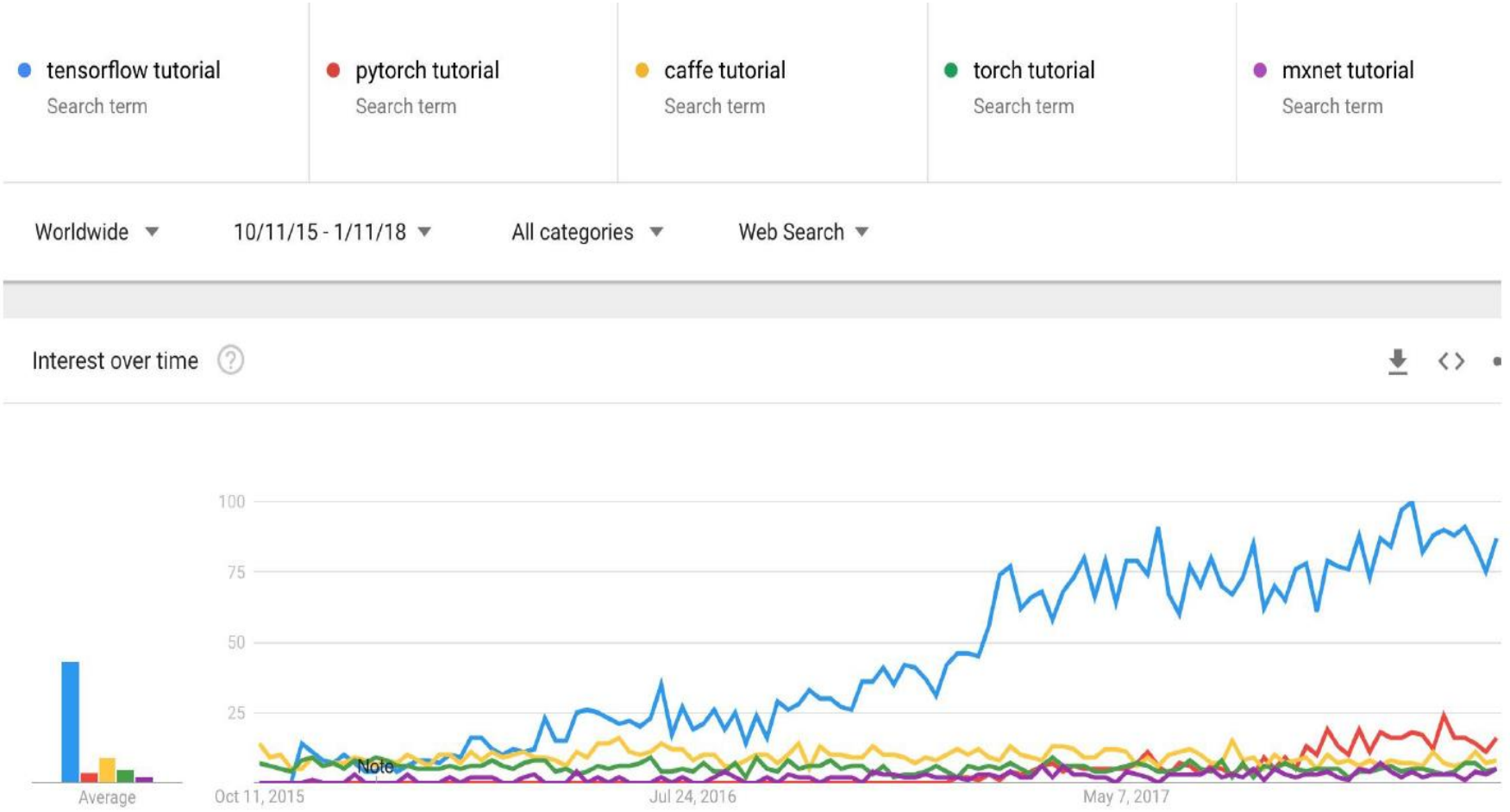
Library	Rank	Overall	Github	Stack Overflow	Google Results
tensorflow	1	10.87	4.25	4.37	2.24
keras	2	1.93	0.61	0.83	0.48
caffe	3	1.86	1.00	0.30	0.55
theano	4	0.76	-0.16	0.36	0.55
pytorch	5	0.48	-0.20	-0.30	0.98
sonnet	6	0.43	-0.33	-0.36	1.12
mxnet	7	0.10	0.12	-0.31	0.28
torch	8	0.01	-0.15	-0.01	0.17
cntk	9	-0.02	0.10	-0.28	0.17
dlib	10	-0.60	-0.40	-0.22	0.02
caffe2	11	-0.67	-0.27	-0.36	-0.04
chainer	12	-0.70	-0.40	-0.23	-0.07
paddlepaddle	13	-0.83	-0.27	-0.37	-0.20
deeplearning4j	14	-0.89	-0.06	-0.32	-0.51
lasagne	15	-1.11	-0.38	-0.29	-0.44
bigdl	16	-1.13	-0.46	-0.37	-0.30
dynet	17	-1.25	-0.47	-0.37	-0.42
apache singa	18	-1.34	-0.50	-0.37	-0.47
nvidia digits	19	-1.39	-0.41	-0.35	-0.64
matconvnet	20	-1.41	-0.49	-0.35	-0.58
tflearn	21	-1.45	-0.23	-0.28	-0.94
nervana neon	22	-1.65	-0.39	-0.37	-0.89
opennn	23	-1.97	-0.53	-0.37	-1.07



DIFERENCIACIÓN DE LOS FRAMEWORKS/LIBRERÍAS

- Especificación del modelo
  - Ficheros de configuración (Caffe, DistBelief, CNTK) vs Programación (Torch/Pytorch, Theano, Tensorflow)
- Modelos de programación, lenguajes de alto nivel:
  - Lua (Torch)
  - Python (Theano, Tensorflow, PyTorch)
  - Otros (DeepLearning4J, Spark MLIB → Java, Scala).

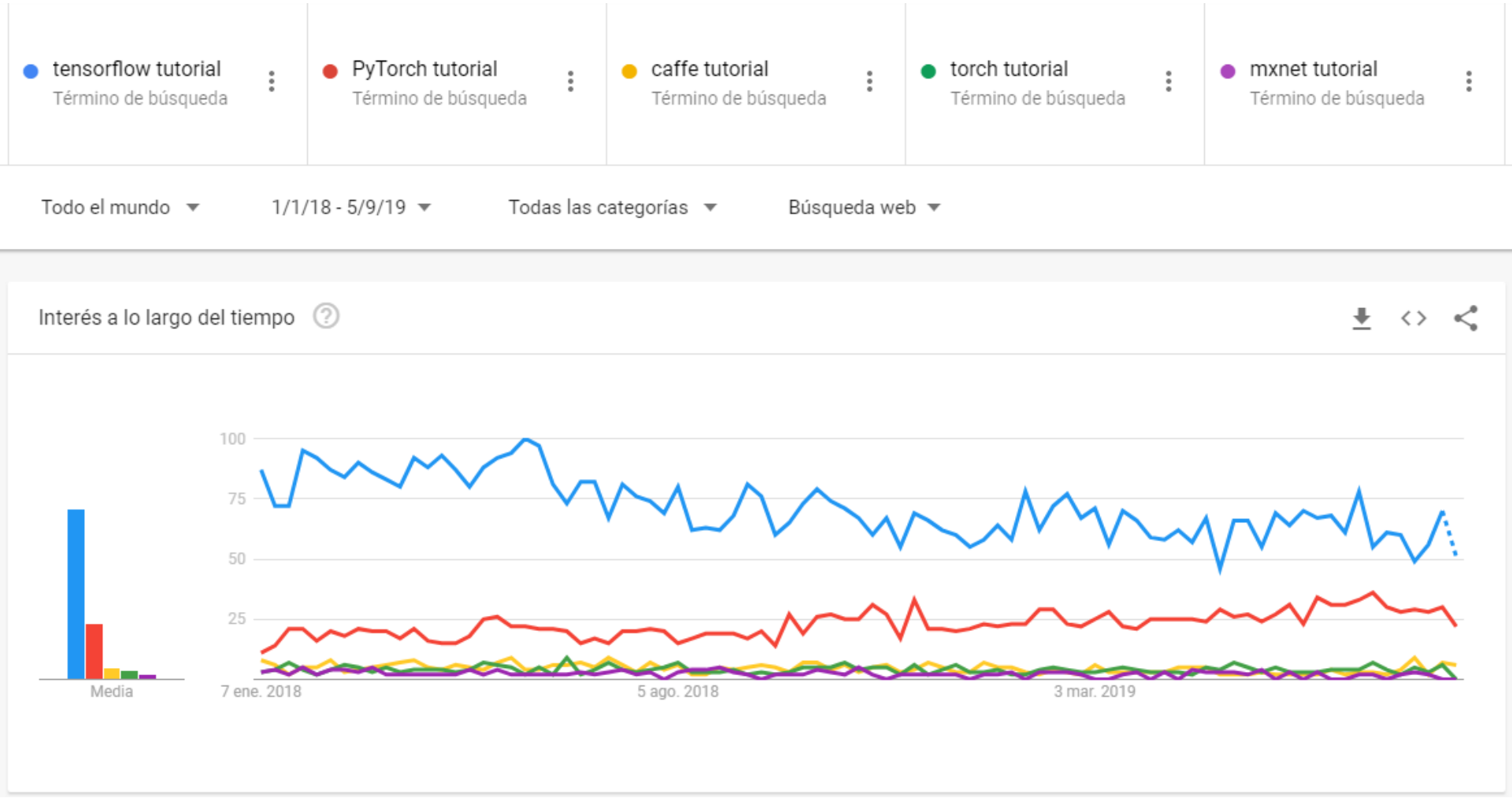
TENDENCIAS (1/3)



Slide from *"Stanford TensorFlow for DL Research course"*

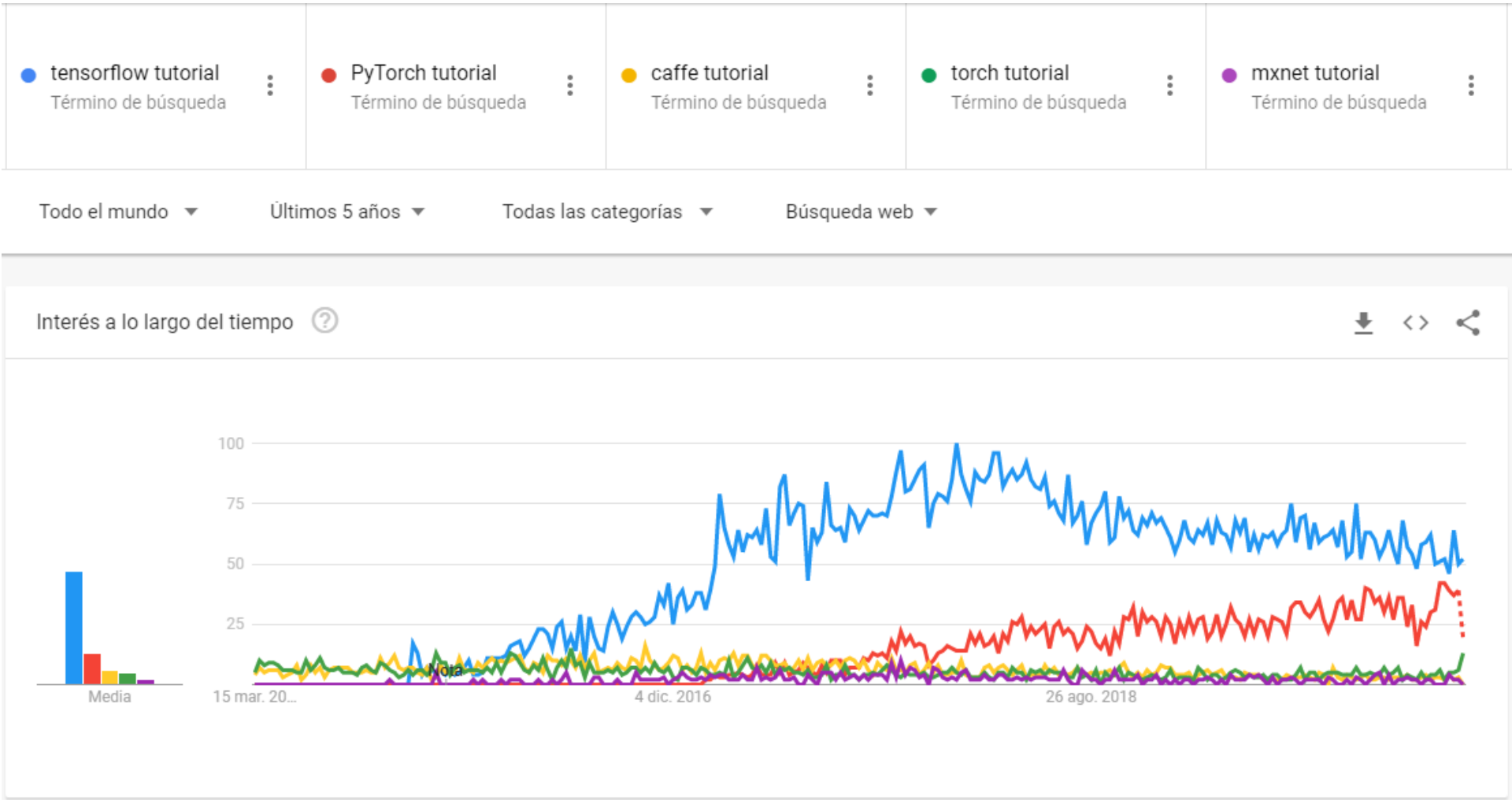
https://trends.google.es/trends/explore?date=2018-01-01%202019-09-05&q=tensorflow%20tutorial,PyTorch%20tutorial,caffe%20tutorial,torch%20tutorial,mxnet%20tutorial

TENDENCIAS (2/3)



https://trends.google.es/trends/explore?date=today%205-y&q=tensorflow%20tutorial,PyTorch%20tutorial,caffe%20tutorial,torch%20tutorial,mxnet%20tutorial

TENDENCIAS (3/3)

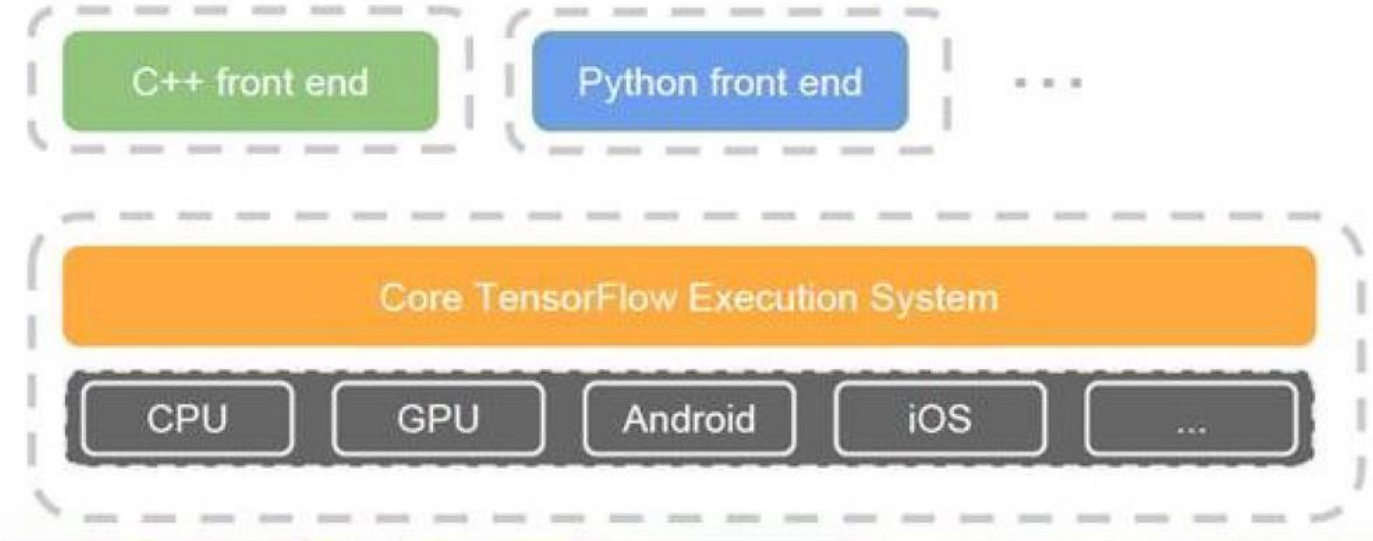


FRAMEWORKS PRINCIPALES

- TensorFlow, Google Brain, 2015 (DistBelief re-escrito)
- Theano, University of Montréal, 2009
- Keras, François Chollet, 2015 (ahora en Google)
- Torch/Pytorch, Facebook AI Research, Twitter, Google DeepMind (FastAI*)
- Caffe, Berkeley Vision and Learning Center (BVLC), 2013

TENSORFLOW

- TensorFlow proporciona primitivas para definir funciones sobre tensores y computar sus derivadas (gradientes)
- Núcleo de bajo nivel (C++/CUDA)
- API Python sencillo para definir el gráfico computacional
- APIs de alto nivel: TF-Learn, TF-Slim, Keras, …



https://www.tensorflow.org/learn

TENSORFLOW (LANGUAGES & RUNTIMES)





Python development

CPU GPU TPU



JavaScript development

CPU GPU TPU



Swift development

CPU GPU TPU



Edge devices

CPU GPU RPi

End-to-end production

CPU GPU TPU

TFX

Validate input data with TF Data Validation

Explore

TFX

Feature engineering with TF Transform

Explore

TFX

Modeling and training

Explore

TFX

Understanding model performance with TF model analysis

Explore

TFX

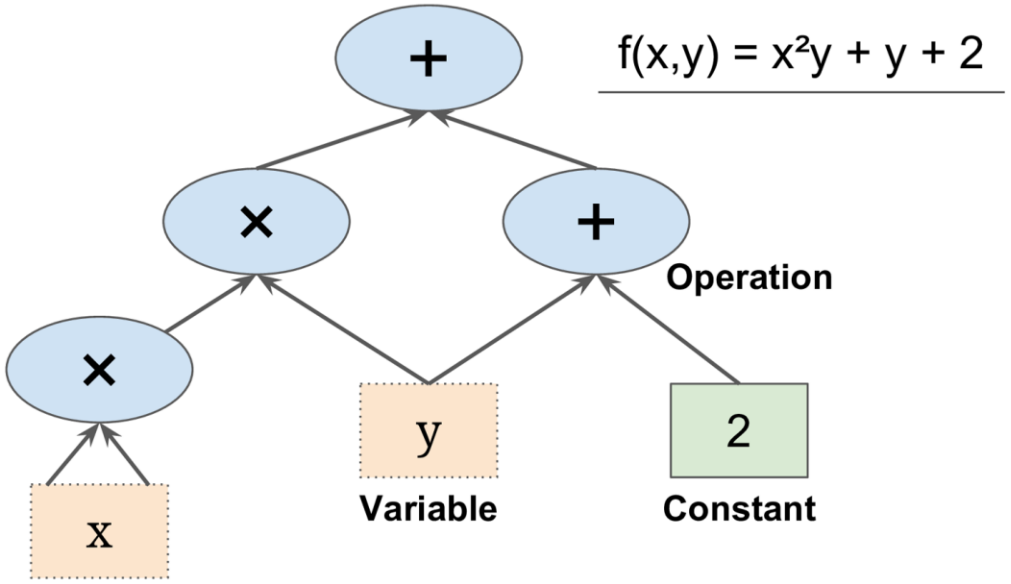
Serve models with a REST API with TF Serving

Explore

TENSORFLOW VERSUS NUMPY

- Muy similares (ambos son librerías de gestión/computación arrays N-dimensionales)
- Numpy no dispone de funciones/métodos para la creación de funciones de tensores y no computa automáticamente sus derivadas
- NumPy no tiene soporte para GPU

TENSORFLOW: MODELO COMPUTACIONAL



$$f(x,y) = x^2y + y + 2$$

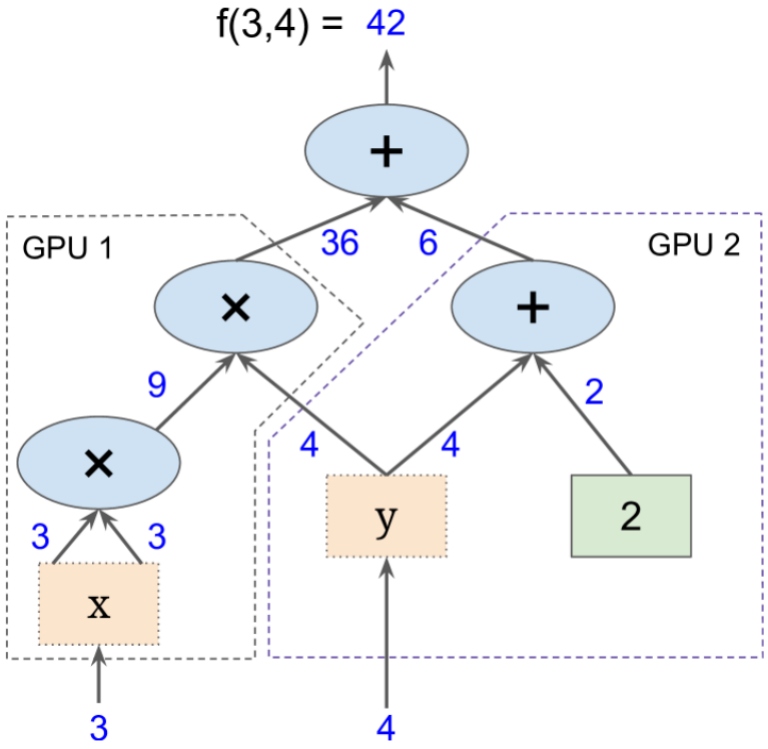
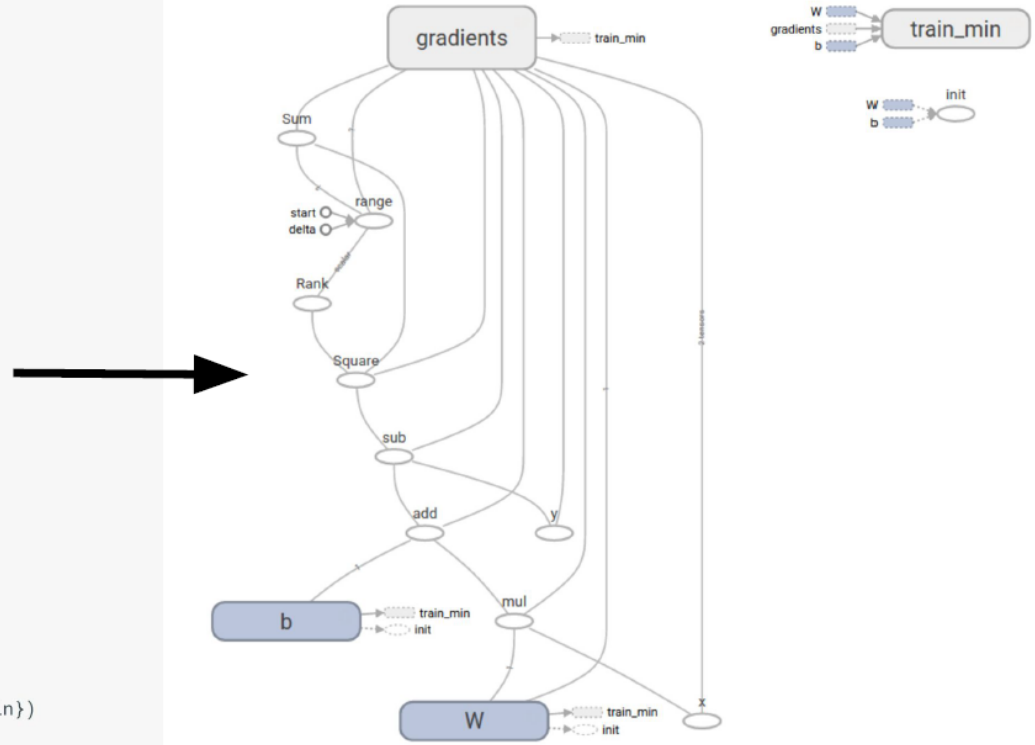


GRÁFICO COMPUTACIONAL: LAZY EVALUATION

```python
import numpy as np
import tensorflow as tf

# Model parameters
W = tf.Variable([.3], tf.float32)
b = tf.Variable([-.3], tf.float32)
# Model input and output
x = tf.placeholder(tf.float32)
linear_model = W * x + b
y = tf.placeholder(tf.float32)
# loss
loss = tf.reduce_sum(tf.square(linear_model - y)) # sum of the squares
# optimizer
optimizer = tf.train.GradientDescentOptimizer(0.01)
train = optimizer.minimize(loss)
# training data
x_train = [1,2,3,4]
y_train = [0,-1,-2,-3]
# training loop
init = tf.global_variables_initializer()
sess = tf.Session()
sess.run(init) # reset values to wrong
for i in range(1000):
  sess.run(train, {x:x_train, y:y_train})

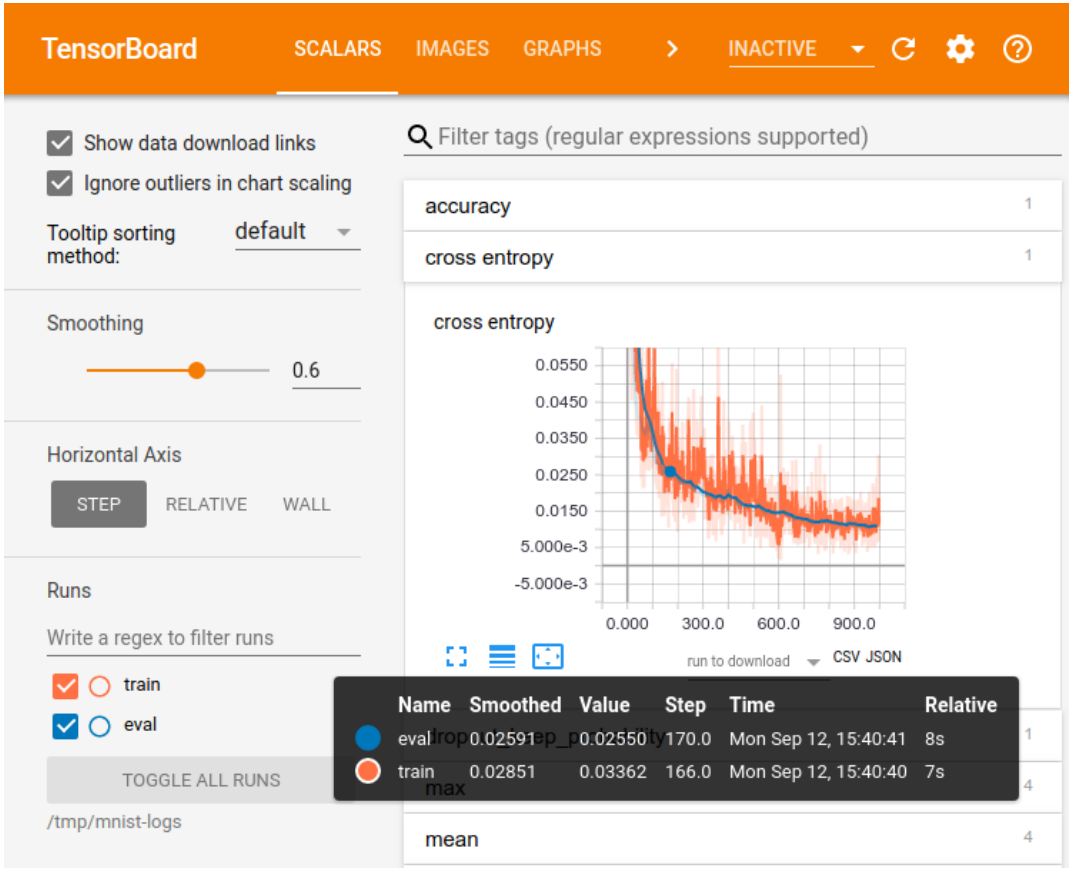
# evaluate training accuracy
curr_W, curr_b, curr_loss  = sess.run([W, b, loss], {x:x_train, y:y_train})
print("W: %s b: %s loss: %s"%(curr_W, curr_b, curr_loss))
```



Slide from *"Stanford TensorFlow for DL Research course"*

TENSORFLOW: VISUALIZACION

- Tensorboard

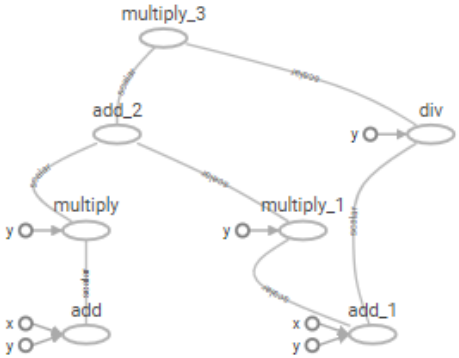


- https://www.tensorflow.org/images/graph_vis_animation.gif

EJERCICIO

- https://github.com/rpastorvargas/deep-learning-samples/blob/master/tensorflow_first.ipynb
- https://github.com/rpastorvargas/deep-learning-samples/blob/master/tensorflow_first_tensorboard.ipynb
  - Si se ejecuta en el entorno local, desde una terminal:

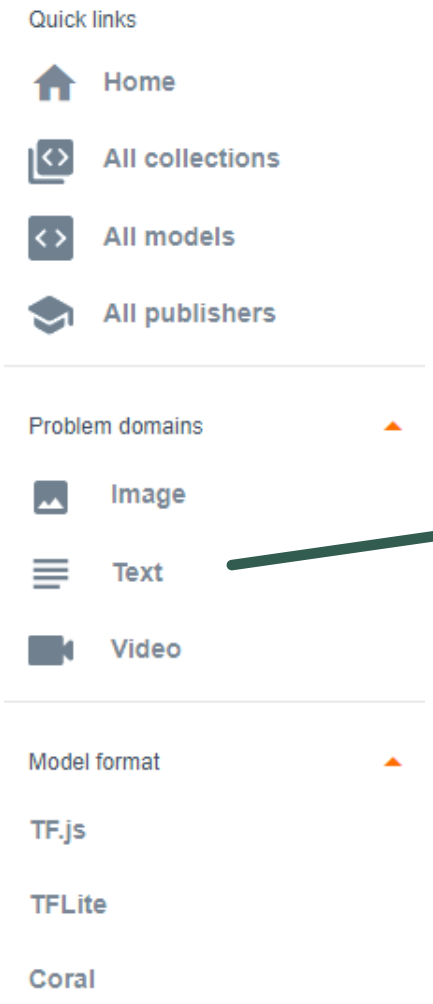
    tensorboard –logdir "output dir" --port 6006





TENSORFLOW HUB

TensorFlow Hub




```
!pip install "tensorflow_hub>=0.6.0"
!pip install "tensorflow>=2.0.0"

import tensorflow as tf
import tensorflow_hub as hub

module_url = "https://tfhub.dev/google/nnlm-en-dim128/2"
embed = hub.KerasLayer(module_url)
embeddings = embed(["A long sentence.", "single-word",
                    "http://example.com"])
print(embeddings.shape)  #(3,128)
```

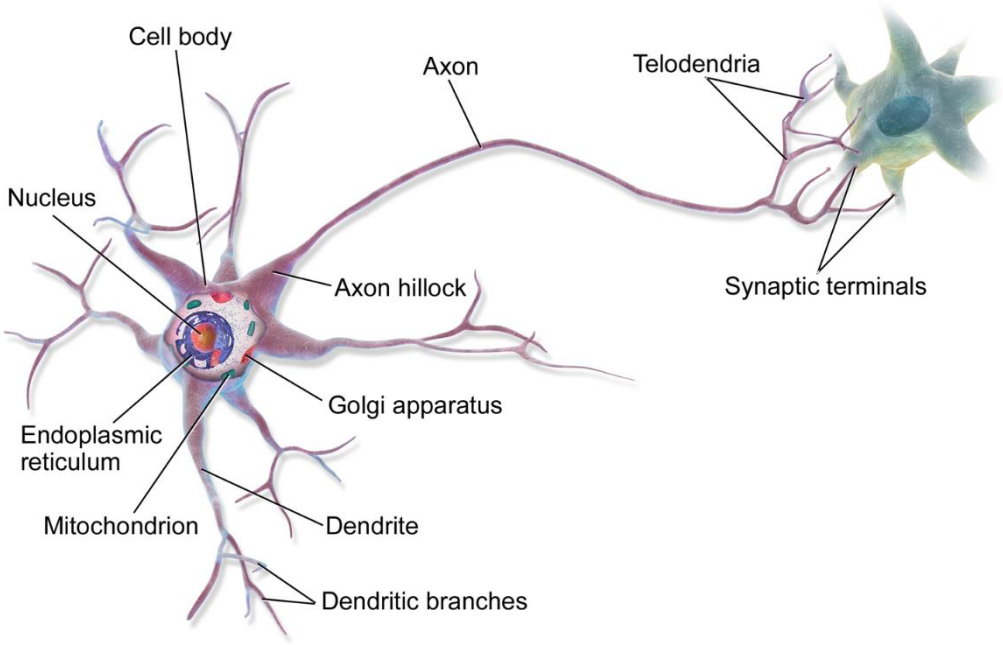
TENSORFLOW: OPERACIONES

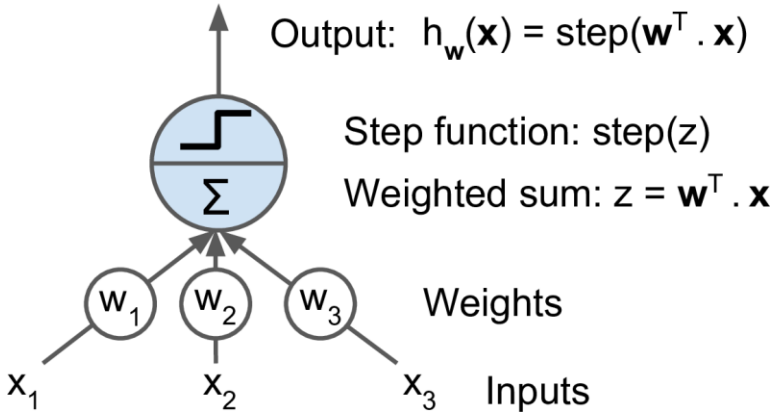
Category	Examples
Element-wise mathematical operations	Add, Sub, Mul, Div, Exp, Log, Greater, Less, Equal, ...
Array operations	Concat, Slice, Split, Constant, Rank, Shape, Shuffle, ...
Matrix operations	MatMul, MatrixInverse, MatrixDeterminant, ...
Stateful operations	Variable, Assign, AssignAdd, ...
Neural network building blocks	SoftMax, Sigmoid, ReLU, Convolution2D, MaxPool, ...
Checkpointing operations	Save, Restore
Queue and synchronization operations	Enqueue, Dequeue, MutexAcquire, MutexRelease, ...
Control flow operations	Merge, Switch, Enter, Leave, NextIteration

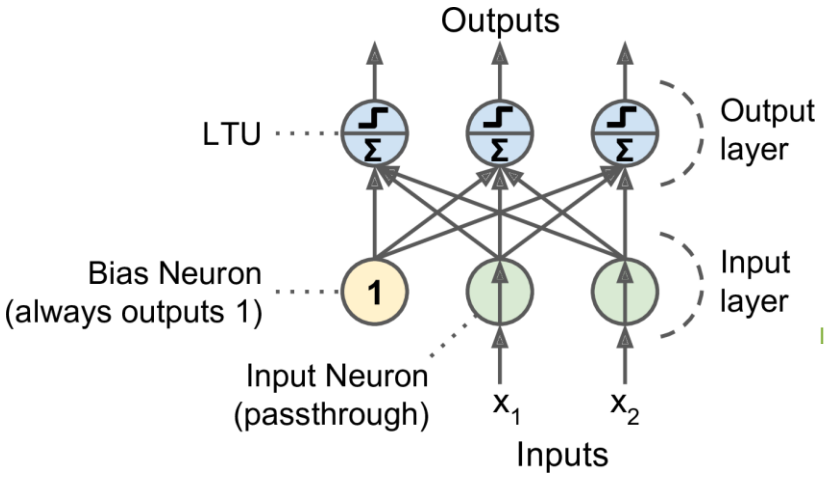
EJERCICIOS

- 09_up_and_running_with_tensorflow.ipynb (https://github.com/ageron/handson-ml)
- https://github.com/rpastorvargas/deep-learning-samples/blob/master/tensorflow_linear_regression.ipynb
- https://github.com/rpastorvargas/deep-learning-samples/blob/master/tensorflow_linear_regression_v2.ipynb
- Qwilabs
  - Data Science en Google Cloud Platform: Machine Learning (https://www.qwiklabs.com/quests/50?locale=es)
    - Aprendizaje automático con TensorFlow (1 hora y 15 min., 7 créditos)

DEEPLEARNING CON TENSORFLOW



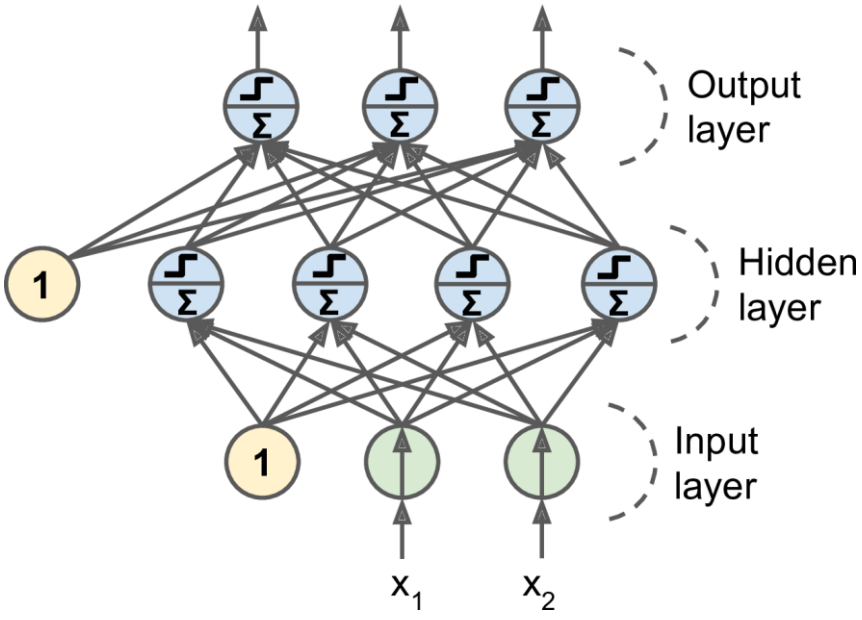


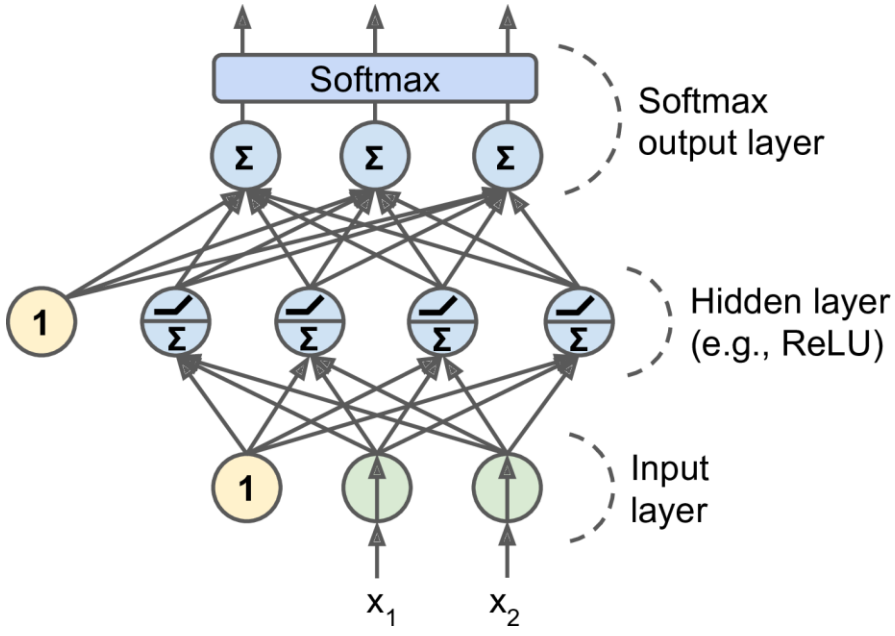


- Perceptron →LTU (Linear Threshold Unit)

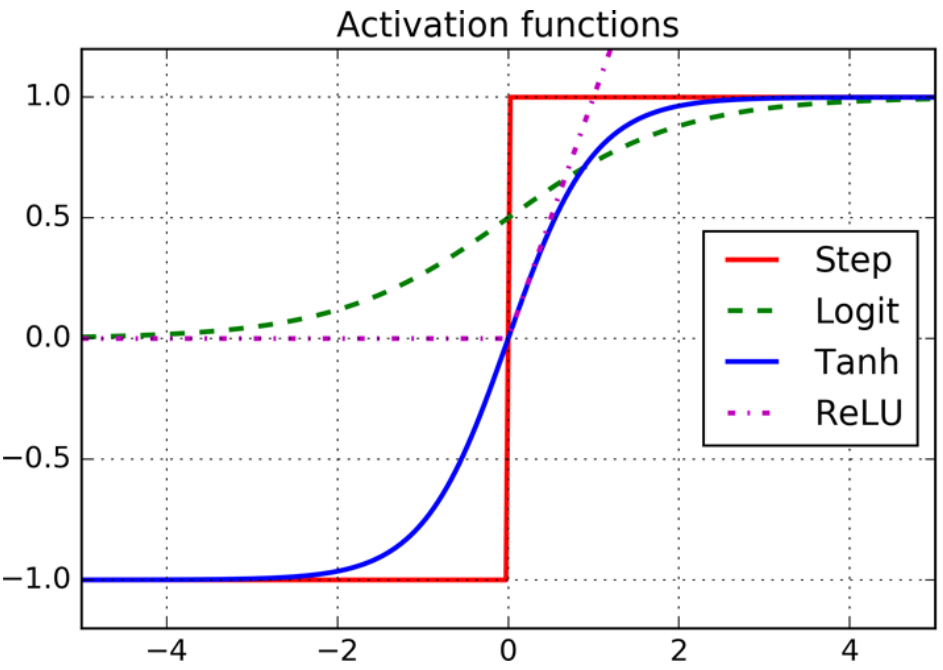
DDN: DEEP NEURAL NETWORK

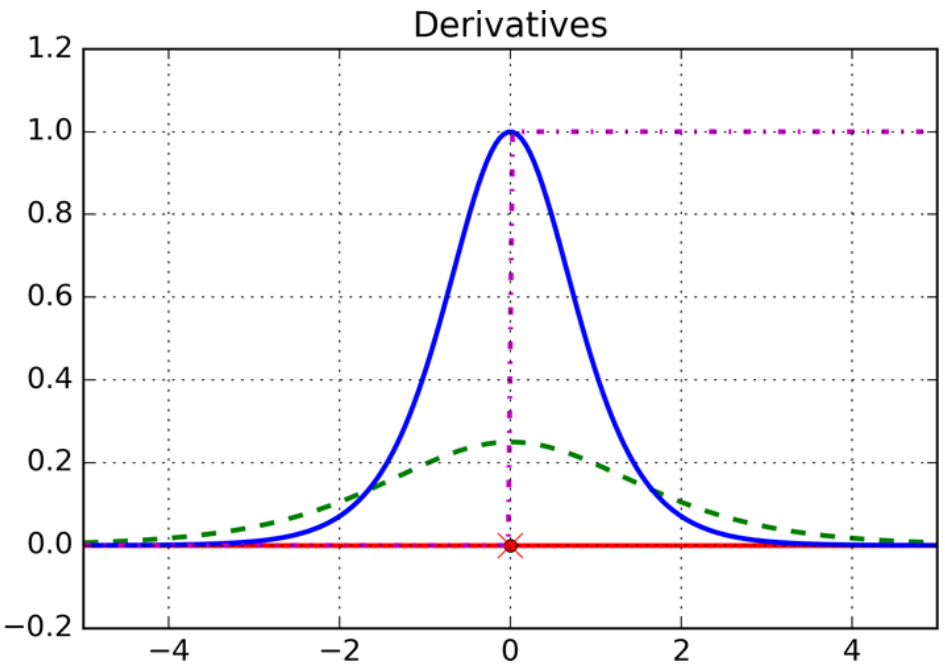
- Multi-Layer Perceptron and Backpropagation (MLP/ANN) →Más de dos capas "hidden" → DNN





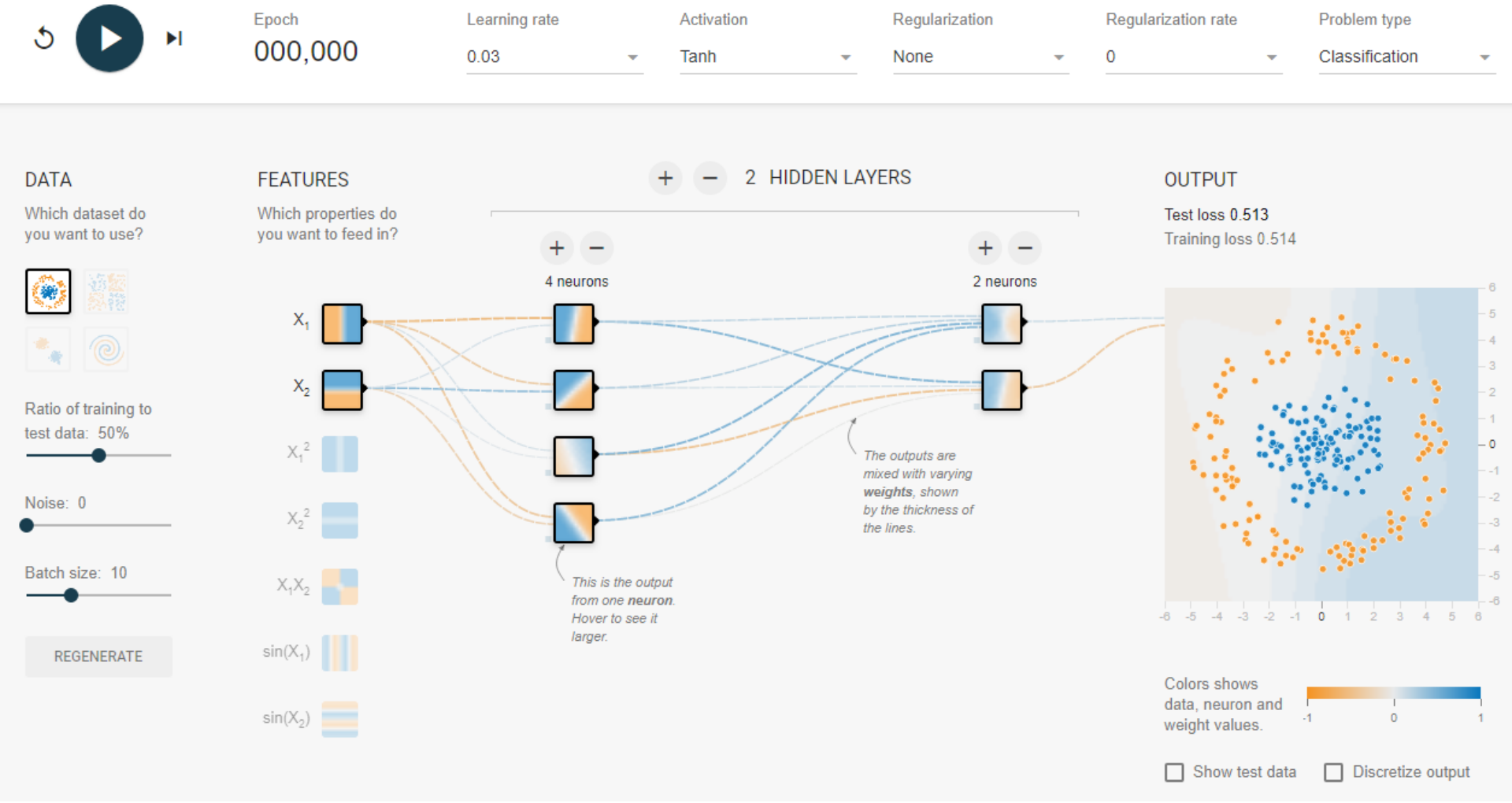
DNN: FUNCIONES DE ACTIVACIÓN





- https://playground.tensorflow.org/

VISUAL DNN



ENTRENAMIENTO DE UNA DDN

- Usando funciones de TensforFlow
- APIs de Alto nivel
  - TF.Learn (compatible con Scikit-Learn)

```
import tensorflow as tf

config = tf.contrib.learn.RunConfig(tf_random_seed=42) # not shown in the config

feature_cols = tf.contrib.learn.infer_real_valued_columns_from_input(X_train)
dnn_clf = tf.contrib.learn.DNNClassifier(hidden_units=[300,100], n_classes=10,
                                         feature_columns=feature_cols, config=config)
dnn_clf = tf.contrib.learn.SKCompat(dnn_clf) # if TensorFlow >= 1.1
dnn_clf.fit(X_train, y_train, batch_size=50, steps=40000)
```

```
from sklearn.metrics import accuracy_score

y_pred = dnn_clf.predict(X_test)
accuracy_score(y_test, y_pred['classes'])
```

ENTRENAMIENTO DE UNA DDN (2/2)

```
import tensorflow as tf

n_inputs = 28*28  # MNIST
n_hidden1 = 300
n_hidden2 = 100
n_outputs = 10
```

```
reset_graph()

X = tf.placeholder(tf.float32, shape=(None, n_inputs), name="X")
y = tf.placeholder(tf.int64, shape=(None), name="y")
```

```
def neuron_layer(X, n_neurons, name, activation=None):
    with tf.name_scope(name):
        n_inputs = int(X.get_shape()[1])
        stddev = 2 / np.sqrt(n_inputs)
        init = tf.truncated_normal((n_inputs, n_neurons), stddev=stddev)
        W = tf.Variable(init, name="kernel")
        b = tf.Variable(tf.zeros([n_neurons]), name="bias")
        Z = tf.matmul(X, W) + b
        if activation is not None:
            return activation(Z)
        else:
            return Z
```

EJERCICIOS

- 10_introduction_to_artificial_neural_networks.ipynb
- mnist_1.0_softmax.py
  - https://github.com/GoogleCloudPlatform/tensorflow-without-a-phd/tree/master/tensorflow-mnist-tutorial
- Data Science en Google Cloud Platform: Machine Learning (https://www.qwiklabs.com/quests/50?locale=es)
  - Aprendizaje automático distribuido con Google Cloud ML (1:30 min., 7 créditos)

- Material adicional:
  - https://www.coursera.org/specializations/advanced-machine-learning-tensorflow-gcp
  - https://www.guru99.com/tensorflow-tutorial.html

THEANO

- Mantenido/Desarrollado por la Universidad de Montreal (MILA Group)
- Pionero en el uso de grafos computacionales.
- Herramienta generalista de aprendizaje máquina -> Lasagne, Keras
- Muy popular en las comunidad investigadora, pero con poco índice de penetración en empresa

- Finalizó su desarrollo a partir de la versión 1.0
  - https://groups.google.com/forum/#!topic/theano-users/7Poq8BZutbY
  - https://github.com/Theano/

- http://videolectures.net/deeplearning2016_lamblin_theano/

COMPONENTES DE ALTO NIVEL

- Python <- {NumPy/SciPy/libgpuarray} <- Theano <- {...}
- libgpuarray: GPU n-dimensional array (objeto en C para CUDA y OpenCL)
- Theano: compilación/manipulación de grafos simbólicos
  - No es un framework/librería de machine learning
- I {...}: Librerías que usan Theano
  - Keras
  - blocks
  - lasagne
  - sklearn-theano
  - PyMC 3
  - theano-rnn
  - Morb

CARACTERÍSTICAS

- Evaluación “Lazy” (rendimiento)
- Soporte para GPUs
- Diferenciación símbolica

MODELOS DESARROLLADOS

- Neural Networks
- Convolutional Neural Networks, AlexNet, OverFeat, GoogLeNet
- RNN, CTC, LSTM, GRU
- NADE, RNADE, MADE
- Autoencoders
- Generative Adversarial Nets
- SVMs
- Muchas variaciones de las anteriores

GRAFO COMPUTACIONAL

- https://github.com/Newmu/Theano-Tutorials

```
import theano
from theano import tensor as T

a = T.scalar()
b = T.scalar()

y = a * b

multiply = theano.function(inputs=[a, b], outputs=y)

print multiply(1, 2) #2
print multiply(3, 3) #9
```

```
import theano
from theano import tensor as T
import numpy as np

trX = np.linspace(-1, 1, 101)
trY = 2 * trX + np.random.randn(*trX.shape) * 0.33

X = T.scalar()
Y = T.scalar()

def model(X, w):
    return X * w

w = theano.shared(np.asarray(0., dtype=theano.config.floatX))
y = model(X, w)

cost = T.mean(T.sqr(y - Y))
gradient = T.grad(cost=cost, wrt=w)
updates = [[w, w - gradient * 0.01]]

train = theano.function(inputs=[X, Y], outputs=cost, updates=updates, allow_input_downcast=True)

for i in range(100):
    for x, y in zip(trX, trY):
        train(x, y)

print w.get_value() #something around 2
```

KERAS

- Librería para redes neuronales escrita en Python
- ¿Porqué usarla?
  - Sencilla para comenzar, y sencilla para avanzar
  - Escrita de forma modular: fácil de expandir
  - Suficientemente potente para escribir modelos serios
- Idea general
  - Preparar los tensores de entrada y salida
  - Crear la primera capa (layer) para manejar el tensor de entrada
  - Crear la última capa (layer) para manejar el tensor de salida (targets)
  - Construir virtualmente cualquier modelo entre estas dos capas (hidden layers)

KERAS

- Simplicidad

TF example:

```
kernel = tf.Variable(tf.truncated_normal([3, 3, 64, 64], type=tf.float32,stddev=1e-1), name='weights')
conv = tf.nn.conv2d(self.conv1_1, kernel, [1, 1, 1, 1], padding='SAME')
biases = tf.Variable(tf.constant(0.0, shape=[64], dtype=tf.float32), trainable=True, name='biases')
out = tf.nn.bias_add(conv, biases)
self.conv1_2 = tf.nn.relu(out, name='block1_conv2')
```

Keras:

```
x = Convolution2D(64, 3, 3, activation='relu', border_mode='same', name='block1_conv2')(x)
```

KERAS, LAYERS

- Keras tiene un número de capas preconstruido extenso. Por ejemplo:
  - Regular dense, MLP type

```
keras.layers.core.Dense(output_dim,
                        init='glorot_uniform',
                        activation='linear',
                        weights=None,
                        W_regularizer=None, b_regularizer=None, activity_regularizer=None,
                        W_constraint=None, b_constraint=None,
                        input_dim=None)
```

  - Recurrent layers, LSTM, GRU, etc.

```
keras.layers.recurrent.GRU(output_dim,
                           init='glorot_uniform', inner_init='orthogonal',
                           activation='sigmoid', inner_activation='hard_sigmoid',
                           return_sequences=False,
                           go_backwards=False,
                           stateful=False,
                           input_dim=None, input_length=None)
```

MÁS CAPAS

- Convolucional 1D, Convolucional 2D, Dropout, Noise, Pooling, Normalization, Embedding

```
keras.layers.convolutional.Convolution1D(nb_filter, filter_length,
        init='uniform',
        activation='linear',
        weights=None,
        border_mode='valid',
        subsample_length=1,
        W_regularizer=None, b_regularizer=None,
        W_constraint=None, b_constraint=None,
        input_dim=None, input_length=None)
```

```
keras.layers.convolutional.Convolution2D(nb_filter, nb_row, nb_col,
        init='glorot_uniform',
        activation='linear',
        weights=None,
        border_mode='valid',
        subsample=(1, 1),
        W_regularizer=None, b_regularizer=None,
        W_constraint=None,
        dim_ordering='th')
```

MÁS CAPACIDADES

- Funciones de activación:
  - Sigmoid, tanh, ReLu, softplus, hard sigmoid, linear
  - Funciones avanzadas de activación implementadas como una capa: LeakyReLu, PReLu, ELU, Parametric Softplus, Thresholded linear y Thresholded Relu
- Funciones objetivo (loss):
  - Error loss: rmse, mse, mae, mape, msle
  - Hinge loss: squared hinge, hinge
  - Class loss: binary crossentropy, categorical crossentropy
- Optimizacion (optimizadores personalizados mediante parámetros):
  - SGD, Adagrad, Adadelta, Rmsprop y Adam
- Soporte de ejecución paralelo: Theano y Tensorflow
  - THEANO FLAGS=mode=FAST RUN,device=gpu, oatX=oat32 python **yournet.py**

KERAS: FORALEZAS Y DEBILIDADES

- Puntos fuertes
  - Es fácil de usar (Python)
  - Se ejecuta sobre Theano y TensorFlow
  - Principios de desarrollo : modularidad, minimalismo, extensible, etc.
  - Disponibilidad de herramientas de visualización (Tensorboard)
  - Está ganando la atención de soporte por parte de Google =>Tensorflow 2.0
- Puntos débiles
  - Menos flexible
  - Menos tipos: no hay modelos RBM, por ejemplo.
  - Menos proyectos disponibles online que Caffe
  - Soporte Multi-GPU no del 100%

MODELOS Y ARQUITECTURAS

- Los modelos pueden ser guardados y recuperados

```
# save as JSON
json_string = model.to_json()
# save as YAML
yaml_string = model.to_yaml()

# model reconstruction from JSON:
from keras.models import model_from_json
model = model_from_json(json_string)

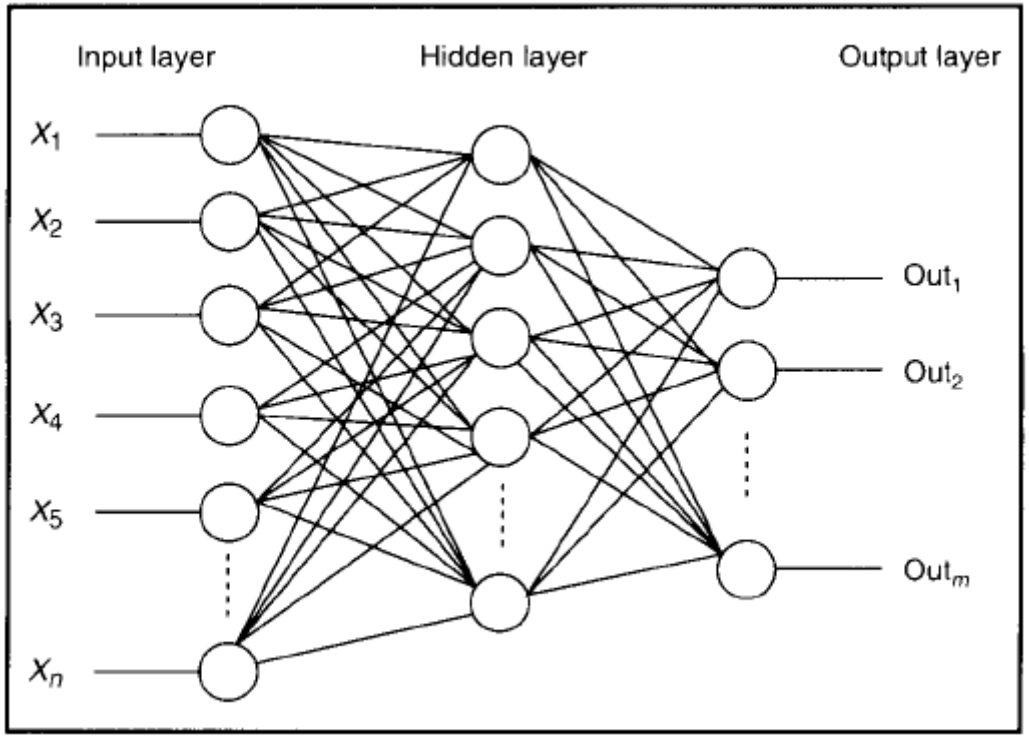
# model reconstruction from YAML
model = model_from_yaml(yaml_string)
```

- Los parámetros del modelo (pesos) pueden ser guardados y recuperados

```
model.save_weights('my_model_weights.h5')
model.load_weights('my_model_weights.h5')
```

TIPOS DE MODELO: SECUENCIAL

- Pila/Cola lineal de capas (MLP)
- https://keras.io/getting-started/sequential-model-guide/



EJERCICIO: PRIMERA RED NEURONAL CON KERAS

- https://machinelearningmastery.com/tutorial-first-neural-network-python-keras/
- Data:
  - https://www.kaggle.com/uciml/pima-indians-diabetes-database/data
- https://github.com/rpastorvargas/deep-learning-samples/blob/master/simple_dnn.ipynb

EJERCICIO

- Reuters newswire topic classification task.
  - https://keras.io/datasets/#reuters-newswire-topics-classification
  - https://martin-thoma.com/nlp-reuters/
  - https://github.com/rpastorvargas/deep-learning-samples/blob/master/reuters_mlp.ipynb

TIPO DE MODELO: API FUNCIONAL

- Se usa para definir modelos complejos: modelos multi-output, grafos acíclicos dirigidos (graph) o modelos con capa compartidas
- https://keras.io/getting-started/functional-api-guide/

```
from keras.layers import Input, Dense
from keras.models import Model

# This returns a tensor
inputs = Input(shape=(784,))

# a layer instance is callable on a tensor, and returns a tensor
x = Dense(64, activation='relu')(inputs)
x = Dense(64, activation='relu')(x)
predictions = Dense(10, activation='softmax')(x)

# This creates a model that includes
# the Input layer and three Dense layers
model = Model(inputs=inputs, outputs=predictions)
model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(data, labels)  # starts training
```

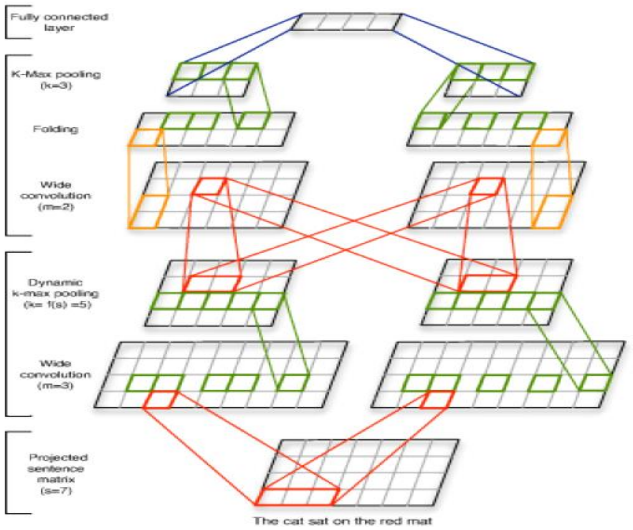
TIPOS DE MODELO: GRAFO (DEPRECATED)

```
vert_test  = np.dstack((X_test[:,:,0], X_test[:,:,3], X_test[:,:,7]))
front_test = np.dstack((-X_test[:,:,1], X_test[:,:,5], X_test[:,:,8]))
side_test  = np.dstack((X_test[:,:,2], X_test[:,:,4], X_test[:,:,6]))
'''
Set things up such that each input takes an axis as input
'''
model = Graph()

model.add_input(name='vert', input_shape=(16501, 3))
model.add_input(name='front', input_shape=(16501,3))
model.add_input(name='side', input_shape=(16501, 3))

'''
Filter for the vertical axis
'''
model.add_node(Convolution1D(nb_filter=20, filter_length=5, activation='relu', input_dim=3,
                             input_length=16501), name='con_vert', input='vert')
model.add_node(Dropout(0.5), name='drop_vert', input='con_vert')
model.add_node(MaxPooling1D(pool_length=10), name='pool_vert', input='drop_vert')
model.add_node(Flatten(), name='flat_vert', input='pool_vert')

'''
Filter for the front axis
'''
model.add_node(Convolution1D(nb_filter=20, filter_length=5, activation='relu', input_dim=3,
                             input_length=16501), name='con_front', input='front')
model.add_node(Dropout(0.5), name='drop_front', input='con_front')
model.add_node(MaxPooling1D(pool_length=10), name='pool_front', input='drop_front')
model.add_node(Flatten(), name='flat_front', input='pool_front')
```



EJERCICIO DE CLASIFICACIÓN DE IMÁGENES

- https://github.com/gggdomi/keras-workshop/
  - ¿Perros o gatos?
  - Requiere pillow para mostrar las imágenes

EJERCICIO DE ANALISIS DE SENTIMIENTO (IMÁGENES)

- https://github.com/yutarochan/HackPSU_DLWorkshop
- Instalar paquetes: opencv, scikit-image, dlib

CAFFE
HTTP://CAFFE.BERKELEYVISION.ORG/

- Open Deep Learning Framework (C++, CUDA)
- Modelos entrenados
- Transparencia de ejecución CPU/GPU

Caffe

build passing | license BSD

Caffe is a deep learning framework made with expression, speed, and modularity in mind. It is developed by Berkeley AI Research (BAIR)/The Berkeley Vision and Learning Center (BVLC) and community contributors.

Check out the project site for all the details like

- DIY Deep Learning for Vision with Caffe
- Tutorial Documentation
- BAIR reference models and the community model zoo
- Installation instructions

and step-by-step examples.

Custom distributions

- Intel Caffe (Optimized for CPU and support for multi-node), in particular Xeon processors (HSW, BDW, SKX, Xeon Phi).
- OpenCL Caffe e.g. for AMD or Intel devices.
- Windows Caffe

MODELO DECLARATIVO

```
layer {
  name: "pool1"
  type: "Pooling"
  pooling_param {
    kernel_size: 2
    stride: 2
    pool: MAX
  }
  bottom: "conv1"
  top: "pool1"
}
```

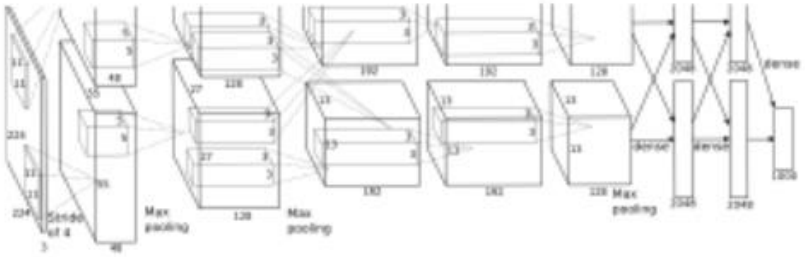
```
layer {
  name: "conv1"
  type: "Convolution"
  bottom: "data"
  top: "conv1"
  param {
    lr_mult: 0
    decay_mult: 0
  }
  convolution_param {
    num_output: 64
    kernel_size: 3
    pad: 1
   }
}
```

```
layer {
  name: "loss"
  type: "SoftmaxWithLoss"
  bottom: "fc8"
  bottom: "label"
  top: "loss"
}
```

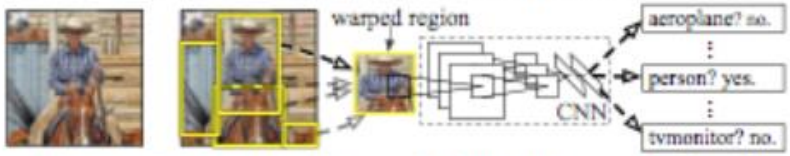
MODELOS DE REFERENCIA:

- Caffe ofrece
  - Definiciones de modelos
  - Parámetros optimizados
  - Pesos pre-entrenados
- Listo para empezar

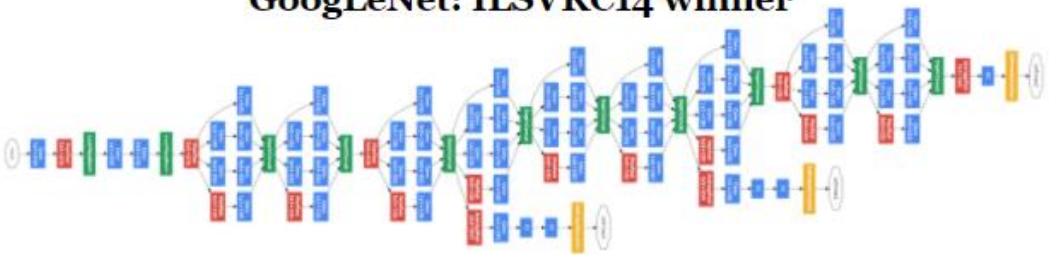
AlexNet: ImageNet Classification



R-CNN: *Regions with CNN features*



GoogLeNet: ILSVRC14 winner



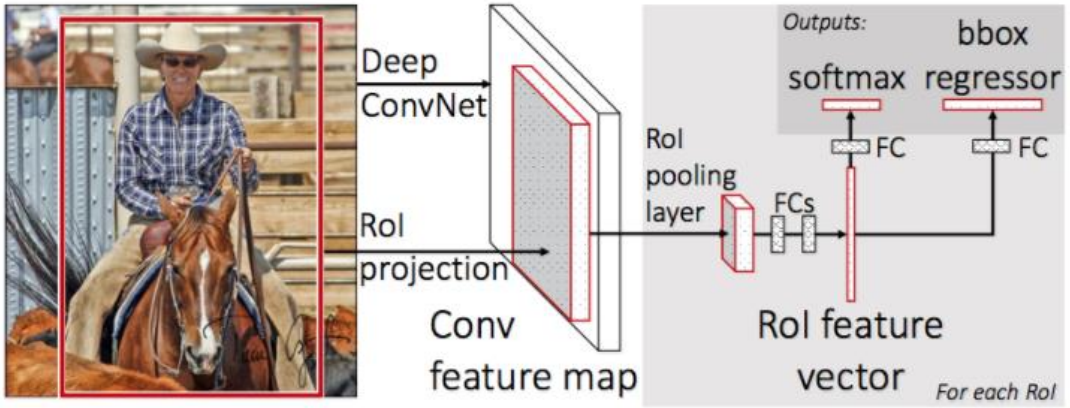
APLICACIONES (I/2)

Object Detection

R-CNNs: Region-based Convolutional Networks

Fast R-CNN

- convolve once
- project + detect



Faster R-CNN

- end-to-end proposals and detection
- image inference in 200 ms
- Region Proposal Net + Fast R-CNN

papers + code online

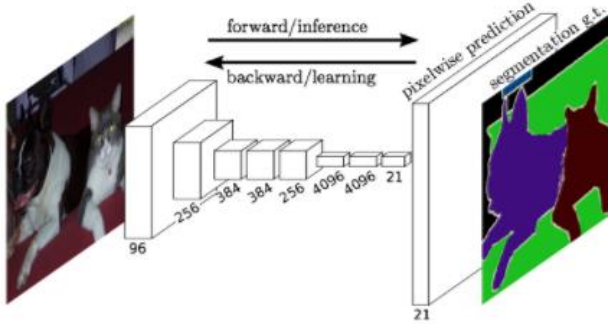
Ross Girshick, Shaoqing Ren, Kaiming He, Jian Sun

APLICACIONES (2/2)

Pixelwise Prediction

Fully convolutional networks for pixel prediction in particular semantic segmentation

- end-to-end learning
- efficient inference and learning
  100 ms per-image prediction
- multi-modal, multi-task



Applications

- semantic segmentation
- denoising
- depth estimation
- optical flow

CVPR'15 paper and code + models



Jon Long & Evan Shelhamer*, Trevor Darrell. CVPR'15*

MÁS APLICACIONES

- **DIY Deep Learning for Vision: A Full-Day Caffe Tutorial**
  - https://docs.google.com/presentation/d/1HxGdeq8MPktHaPb-rlmYYQ723iWzq9ur6Gjo71YiG0Y/edit#slide=id.gc2fcdcce7_216_0

DEMO

- http://places2.csail.mit.edu/demo.html
- http://demo.vislab.berkeleyvision.org/
- http://demo.caffe.berkeleyvision.org/



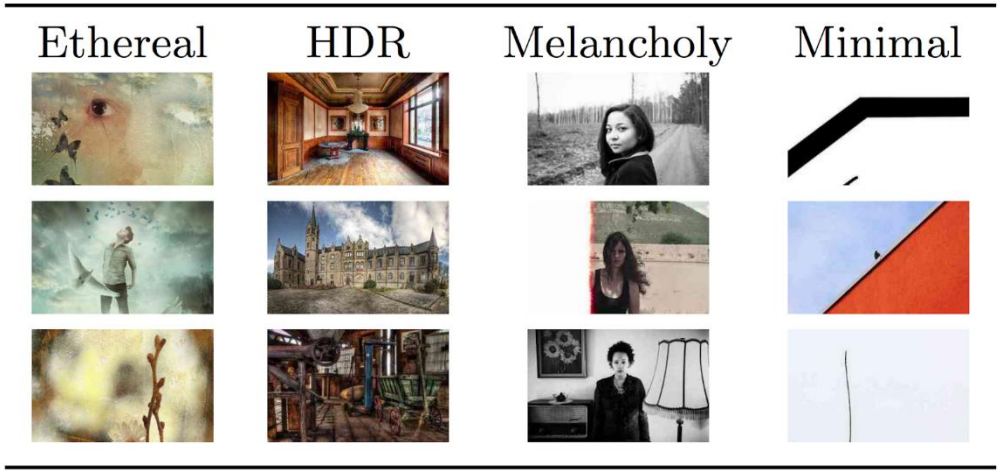
Maximally accurate | Maximally specific

cat	1.80727
domestic cat	1.74727
feline	1.72787
tabby	0.99133
domestic animal	0.78542



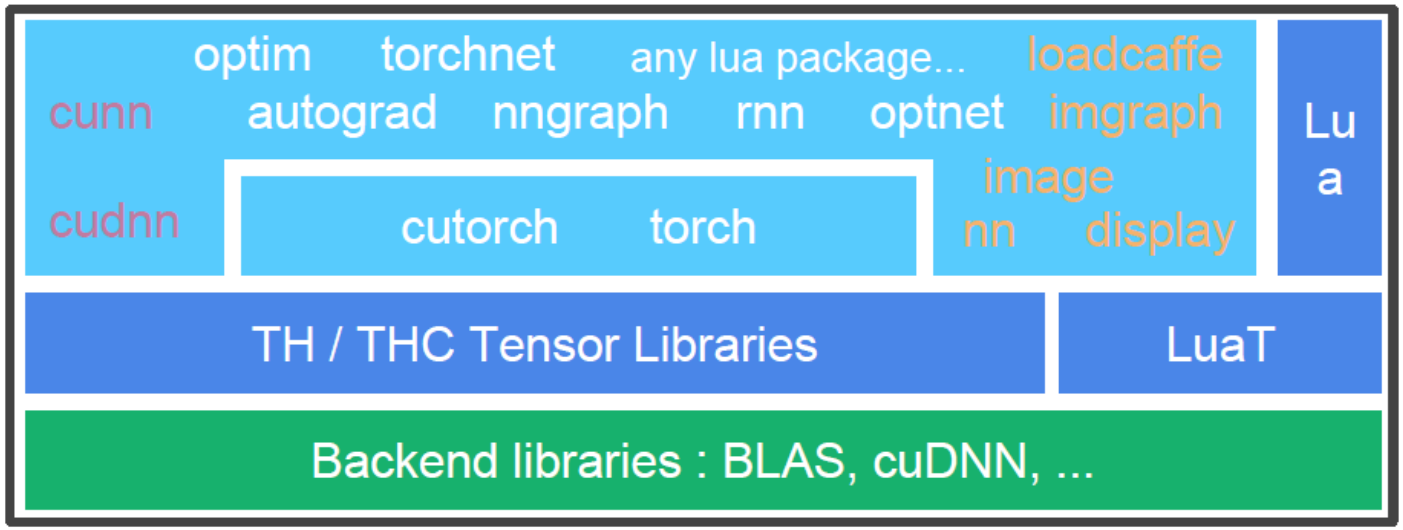
Predictions:

- **Type of environment:** outdoor
- **Semantic categories:** skyscraper:0.69, tower:0.16, office_building:0.11,
- **SUN scene attributes:** man-made, vertical components, natural light, open area, nohorizon, glossy, metal, wire, clouds, far-away horizon



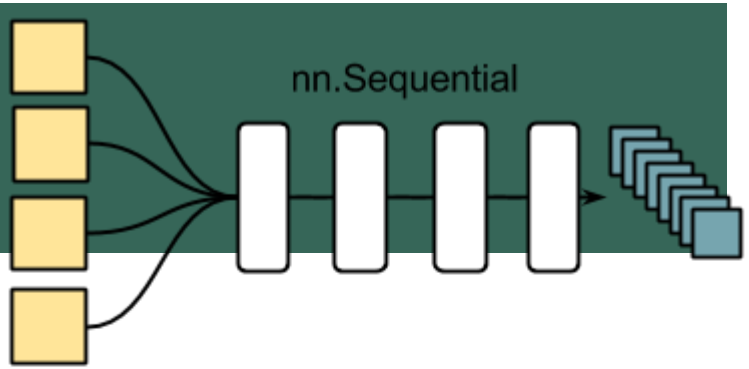
TORCH

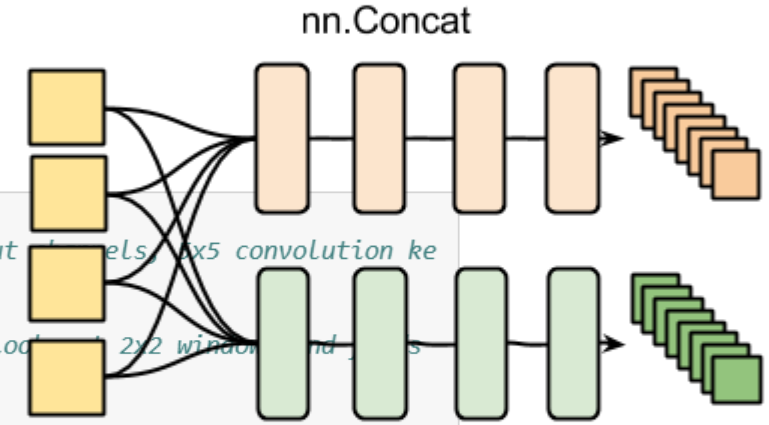
- Lenguajes :
- ➢ C / CUDA backend
- ➢ Lua frontend, ejecutándose en LuaJIT (Lua es rápido y tiene una interface muy directa con C)

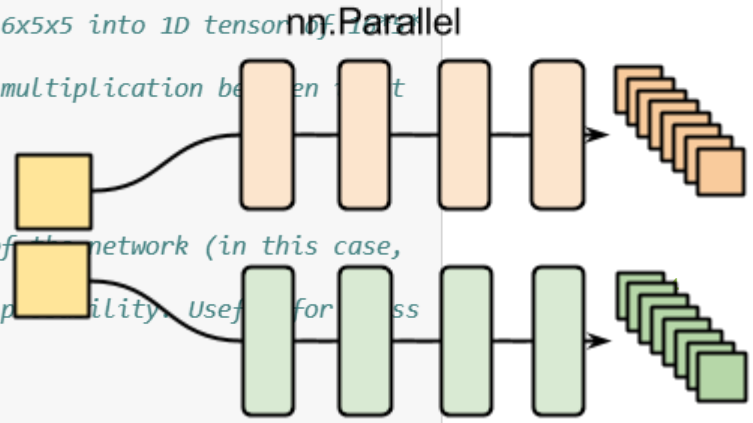


RECURSOS PARA COMENZAR

- Referencia:
  - https://github.com/torch/torch7/wiki/Cheatsheet
- Tutorial:
  - https://github.com/soumith/cvpr2015/blob/master/Deep%20Learning%20with%20Torch.ipynb



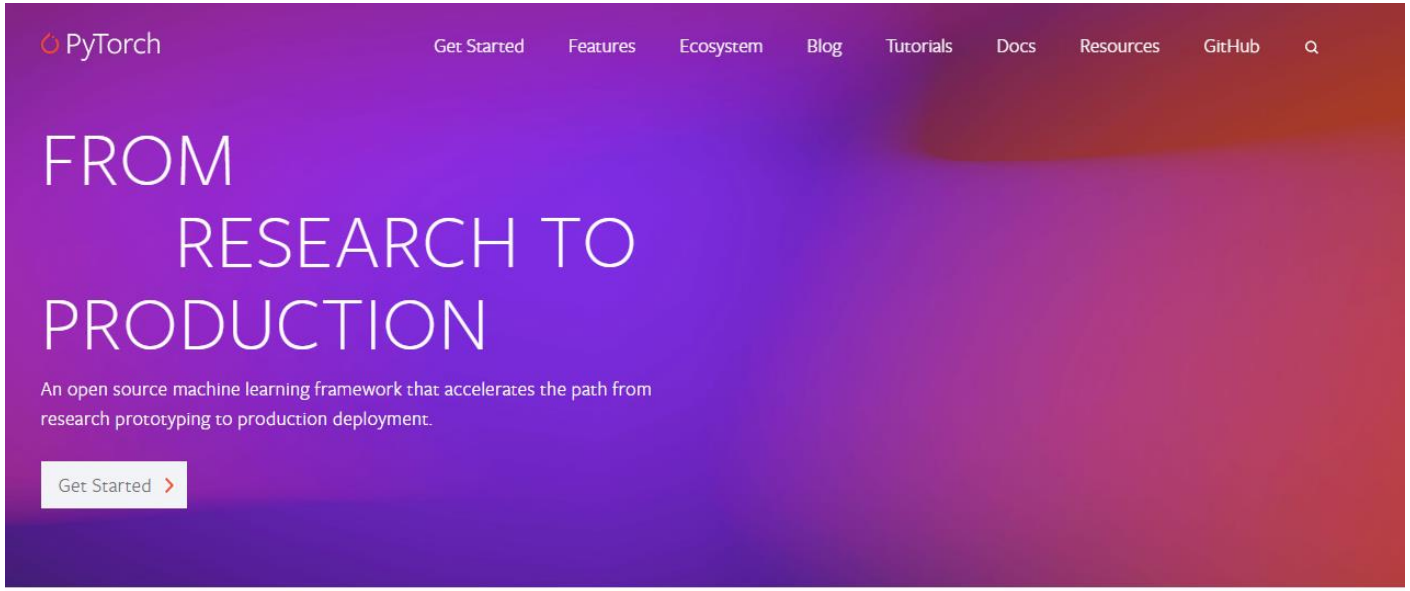





```
net = nn.Sequential()
net:add(nn.SpatialConvolution(1, 6, 5, 5)) -- 1 input image channel, 6 output channels, 5x5 convolution ke
rnel
net:add(nn.ReLU())                         -- non-linearity
net:add(nn.SpatialMaxPooling(2,2,2,2))     -- A max-pooling operation that looks at 2x2 windows and finds
the max.
net:add(nn.SpatialConvolution(6, 16, 5, 5))
net:add(nn.ReLU())                         -- non-linearity
net:add(nn.SpatialMaxPooling(2,2,2,2))
net:add(nn.View(16*5*5))                   -- reshapes from a 3D tensor of 16x5x5 into 1D tensor of 16*5*
5
net:add(nn.Linear(16*5*5, 120))            -- fully connected layer (matrix multiplication between input
and weights)
net:add(nn.ReLU())                         -- non-linearity
net:add(nn.Linear(120, 84))
net:add(nn.ReLU())                         -- non-linearity
net:add(nn.Linear(84, 10))                 -- 10 is the number of outputs of the network (in this case,
10 digits)
net:add(nn.LogSoftMax())                   -- converts the output to a log-probability. Useful for class
ification problems

print('Lenet5\n' .. net:__tostring());
```

PYTORCH



KEY FEATURES & CAPABILITIES

See all Features

TorchScript

TorchScript provides a seamless transition between eager mode and graph mode to accelerate the path to production.

Distributed Training

Scalable distributed training and performance optimization in research and production is enabled by the torch.distributed backend.

Python-First

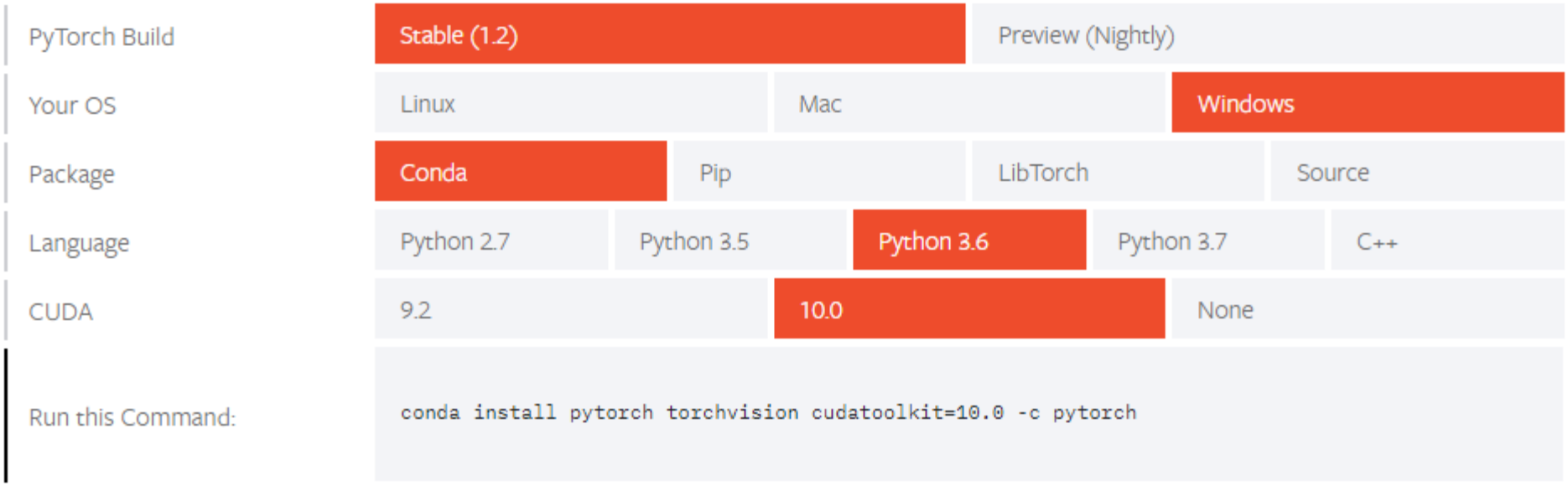
Deep integration into Python allows popular libraries and packages to be used for easily writing neural network layers in Python.

Tools & Libraries

A rich ecosystem of tools and libraries extends PyTorch and supports development in computer vision, NLP and more.

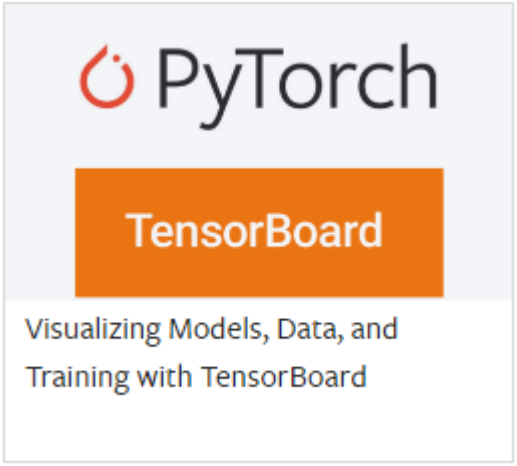
COMENZANDO CON PYTORCH

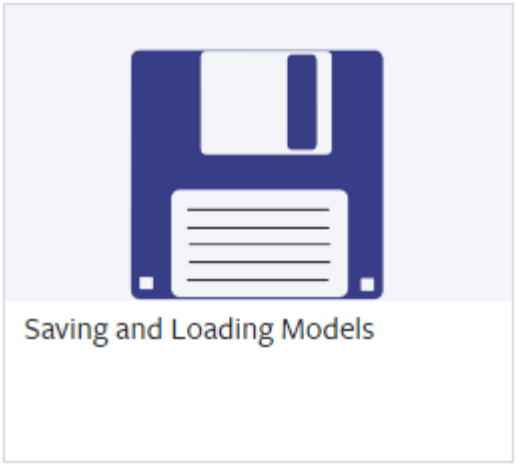
- Instalación:
  - conda install pytorch torchvision cpuonly -c pytorch (Windows, No CPU)



EJERCICIO

- https://pytorch.org/tutorials/
  - http://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html
    - http://pytorch.org/tutorials/beginner/blitz/tensor_tutorial.html
    - http://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html
    - http://pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html
    - http://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html







- https://docs.fast.ai/

FAST.AI

```
path = untar_data(URLs.MNIST_SAMPLE)
data = ImageDataBunch.from_folder(path)
learn = cnn_learner(data, models.resnet18, metrics=accuracy)
learn.fit(1)
```

Total time: 00:09

epoch	train_loss	valid_loss	accuracy
1	0.128580	0.082647	0.973503

https://docs.fast.ai/training.html

Application fields

The fastai library allows you to train a `Model` on a certain `DataBunch` very easily by binding them together inside a `Learner` object. This module regroups the tools the library provides to help you preprocess and group your data in this format.

collab

This submodule handles the collaborative filtering problems.

tabular

This sub-package deals with tabular (or structured) data.

text

This sub-package contains everything you need for Natural Language Processing.

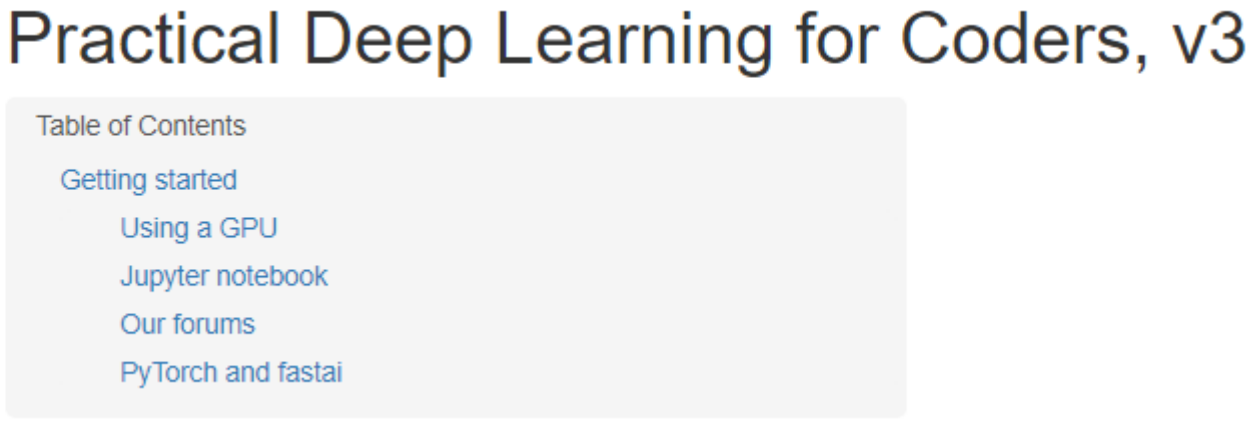
vision

This sub-package contains the classes that deal with Computer Vision.

EJERCICIO

- https://course.fast.ai/





- https://github.com/fastai/course-v3
  - pip install fastai (no está disponible en Conda)
  - https://github.com/fastai/course-v3/blob/master/nbs/dl1/lesson1-pets.ipynb

APLICACIONES DEEP LEARNING

- Vision (reconocimiento imágenes, detección movimiento, etc.)
- Speech Recognition
- NLP
  - Y muchas mas…

- https://dzone.com/articles/machine-learning-real-world-applications
- https://medium.com/@vratulmittal/top-15-deep-learning-applications-that-will-rule-the-world-in-2018-and-beyond-7c6130c43b01
- https://www.forbes.com/sites/kevinmurnane/2016/04/01/thirteen-companies-that-use-deep-learning-to-produce-actionable-results/
- https://www.analyticsvidhya.com/blog/2017/02/6-deep-learning-applications-beginner-python/